

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	A. Likithasri	Reg. No.:	192511003
Section / Batch:	2	Date:	17/07/2026

Code of Conduct

I, A. Likithasri (192511003) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Likitha

Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

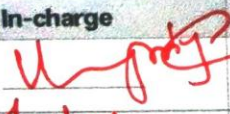
Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:			100 Marks

Annexure A – Analytical Problem Solving

S.No	REG NO	NAME	ANALYTICAL PROBLEM SOLVING
2	192511003	ARREDDUL A LIKTHASRI	1. Perform the following operations in Circular Queue having a size of 5. Enqueue(20), Enqueue(10), Enqueue(30), Dequeue(), Enqueue(80), Dequeue(), Enqueue(90), Enqueue(100), Dequeue(), Dequeue(), Enqueue(60), Enqueue(90). Finally display the front and rear elements with its positions. Discuss in detail. 2. Implement Binary searching Algorithm for searching an element and its position having the following 10 elements. Elements are 66, 111, 212, 323, 432, 543, 643, 649, 777, 888. Case 1: Key element = 111, Case 2: Key element=888, Case 3: Key element = 999. Analyse the concept in detail.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: 18/7/26	Date: _____	Date: _____

1) Perform the following operations in circular queue (Size = 5)

Problem understanding:

A circular queue is a linear data structure in which the last position is connected to the first position. It efficiently utilizes memory by reusing empty spaces created after deletion.

- Queue size = 5
- Index positions = 0, 1, 2, 3, 4

Algorithm

1. Initialize front = Rear = -1.
2. Enqueue: Insert the element at the rear. If the rear reaches the last index, wrap it to index 0.
3. Dequeue: Remove the front element and move the front to the next position.
4. Display the final queue, front and rear.

Step by step execution:

Step	Operation	Front	Rear
1.	Enqueue(20)	0	0
2.	Enqueue(10)	0	1
3.	Enqueue(30)	0	2
4.	Dequeue() - remove 20	1	2
5.	Enqueue(80)	1	3
6.	Dequeue() - remove 10	2	3
7.	enqueue(90)	2	4
8.	enqueue(100)	2	0
9.	Dequeue() - remove 30	3	0
10.	Dequeue() - remove 80	4	0

11.	Enqueue (60)	4	1
12.	Enqueue (90)	4	2

Final queue:

Index	0	1	2	3	4
Value	100	60	90	-	90

queue order (Front - rear) :

90 $\rightarrow$ 100 $\rightarrow$ 60 $\rightarrow$ 90

First

Final Result:

- front element = 90
- front position (index) = 4
- Rear element = 90
- Rear Position (Index) = 2

Analysis:

- Circular queue avoids memory wastage by reusing empty positions.
- Enqueue time Complexity = $O(1)$
- Dequeue time Complexity = $O(1)$

Conclusion

Circular queue provides efficient memory utilization compared to a linear queue. The final front element is 90 at index 4, and the rear element is 90 at index 2.

Implement Binary Search algorithm

2) Problem understanding:

Binary Search is an efficient searching algorithm that works only on a sorted array.

Array:

Index	0	1	2	3	4	5
Element	66	111	212	323	432	543

Algorithm:

1. Set $low = 0$ and $high = n - 1$.
2. Find $Mid = (low + high) / 2$.
3. Compare the key with the middle element.
4. If equal, the element is found.
5. If the key is smaller, search the left half.
6. If the key is larger, search the right half.
7. Repeat until the element is found or $low > high$.

Case 1: key = 111

Iteration	low	high	mid	element	Result
1	0	5	2	212	Search left
2	0	1	0	66	Search right
3	1	1	1	111	Found

Result: element 111 is found at index 1 (2nd position)

Case 2: key = 888

iteration	low	high	mid	element	Result
1	0	9	4	432	Search Right
2	5	9	7	649	Search Right
3	8	9	8	777	Search Right
4	9	9	9	888	found

Result: element 888 is found at index 9 (10th position)

Case 3: key = 999

iteration	low	high	mid	element	result
1	0	9	4	432	search right
2	5	9	7	649	Search right
3	8	9	8	777	Search right
4	9	9	9	888	Search right
5	10	9	—	—	not found

Result: Element 999 is not present in the array.

Analysis:

Advantages:

- fast searching for sorted data
- Requires fewer comparisons than linear search.

Time complexity:

- Best case: $O(1)$
- Average case: $O(\log n)$
- Worst case: $O(\log n)$

